[논문리뷰] Hybrid Transformers for Music Source Separation (HT Demucs)

arXiv 2022. [Paper] [Github]
Simon Rouard, Francisco Massa, Alexandre Defossez
Meta AI
15 Nov 2022

Introduction

2015년 Signal Separation Evaluation Campaign (SiSEC) 이후 MSS 커뮤니티는 노래를 드럼, 베이스, 보컬 및 기타 (기타 모든 악기)의 4가지로 분리하기 위해 supervised model을 학습시키는 task에 주로 중점을 두었다. MSS를 벤치마킹하는 데 사용되는 레퍼런스 데이터셋는 두 가지 버전(HQ 및 non-HQ)의 150곡으로 구성된 MUSDB18이다. 학습셋은 87곡으로 구성되어 있으며, Transformer 기반 아키텍처가 비전 또는 NLP와 같이 널리 성공하고 채택된 다른 딥 러닝 기반 task에 비해 상대적으로 적게 채택되었다. Source separation은 짧은 컨텍스트 또는 긴 컨텍스트를 입력으로 사용하는 task이다. Conv-Tasnet은 로컬한 acoustic feature만 사용하여 separation을 수행하기 위해 약 1초의 컨텍스트를 사용한다. 반면 Demucs는 최대 10초의 컨텍스트를 사용할 수 있으므로 입력의 모호성을 해결하는 데 도움이 될 수 있다. 본 논문에서는 Transformer 아키텍처가 이러한 컨텍스트를 활용하는 데 어떻게 도움이 될 수 있는지, 이를 학습시키는 데 필요한 데이터 양을 연구하는 것을 목표로 한다.

먼저 새로운 아키텍처인 **Hybrid Transformer Demucs (HT Demucs)**를 제시한다. 이 아키텍처는 원래 Hybrid Demucs 아키텍처의 가장 안쪽 layer를 Transformer layer로 대체하고 도메인 내 self-attention과 도메인 간 cross-attention을 사용하여 시간 표현과 스펙트럼 표현에 모두 적용된다. Transformer는 일반적으로 데이터가 부족하기 때문에 MUSDB 데이터셋 위에 800개의 노래로 구성된 내부 데이터셋을 활용한다.

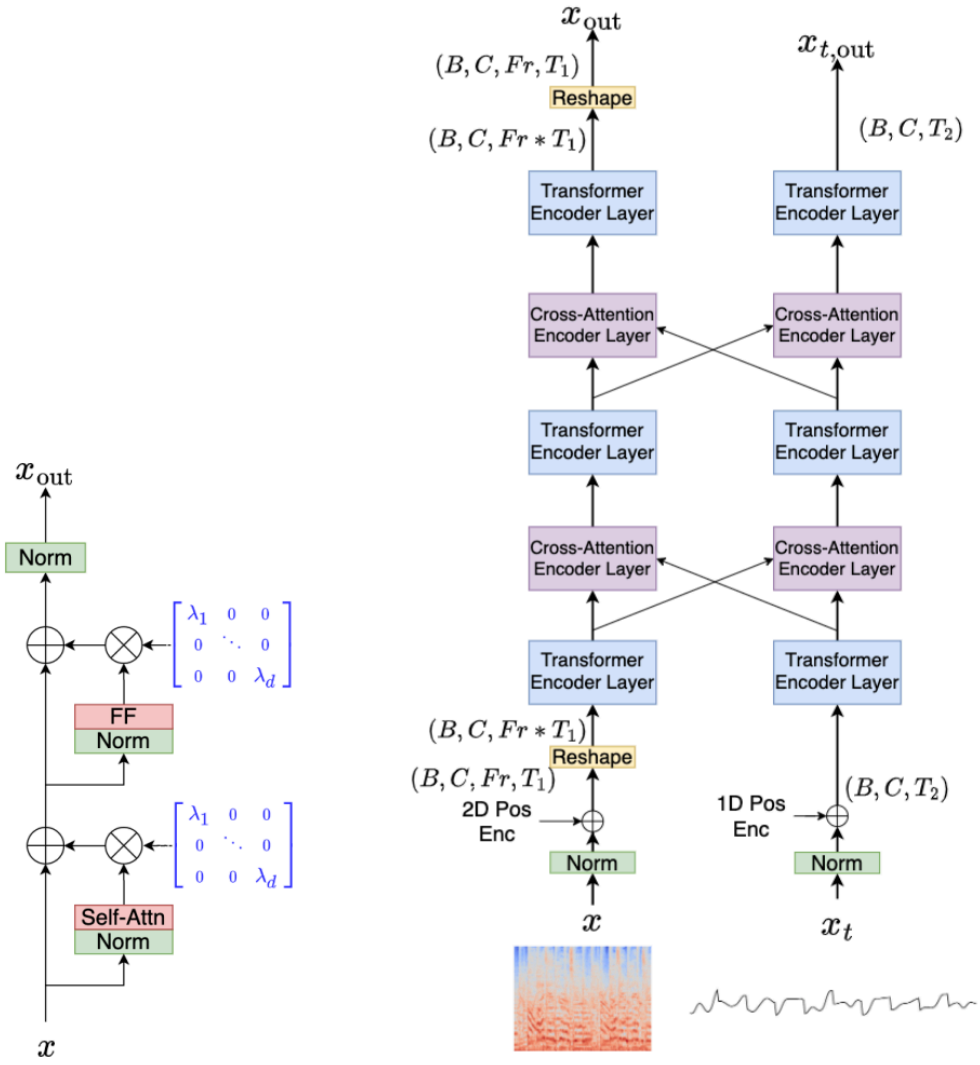
또한 저자들은 다양한 설정 (깊이, 채널 수, 컨텍스트 길이, 확장 등)으로 이 새로운 아키텍처를 광범위하게 평가하였다. 특히 기본 Hybrid Demucs 아키텍처보다 0.35dB 향상되었음을 보여준다. 마지막으로 학습 중 메모리 문제를 극복하기 위해 Locally Sensitive Hashing을 기반으로 하는 sparse kernel을 사용하여 컨텍스트 지속 시간을 늘리고 fine-tuning하여 MUSDB의 테스트셋에서 최종 SDR 9.20dB를 달성하였다.

Architecture

본 논문은 Hybrid Demucs를 기반으로 한 Hybrid Transformer Demucs 모델을 도입한다. 원래의 Hybrid Demucs 모델은 두 개의 U-Net으로 구성된다. 하나는 시간 도메인(temporal convolution 사용)과 스펙트로그램 도메인(주파수 축에 대한 convolution 사용)에 있다. 각 U-Net은 5개의 인코더 layer과 5개의 디코더 layer로 구성된다. 5번째 인코더 layer 이후 두 표현은 동일한 모양을 가지며 공유된 6번째 layer로 이동하기 전에 합산된다. 마찬가지로 첫 번째 디코더 layer는 공유되며 그 출력은 시간 및 스펙트럼 분기 모두에 전송된다. 스펙트럼 분기의 출력은 시간 분기의 출력과 합산되기 전에 iSTFT를 사용하여 파형으로 변환되어 모델의 실제 예측을 준다.

Hybrid Transformer Demucs는 원래 아키텍처의 가장 바깥쪽 4개 layer를 그대로 유지하고 local attention과 bi-LSTM을 포함하여 인코더 및 디코더의 가장 안쪽 2개 layer를 Cross-domain Transformer Encoder로 대체한다. 스펙트럼 분기의 2D 신호와 파형 분기의 1D 신호를 병렬로 처리한다. 시간 표현과 스펙트럼 표현을 정렬하기 위해 모

델 파라미터 (STFT window 및 hop 길이, stride, padding 등)를 신중하게 조정해야 했던 원래의 Hybrid Demucs와 달리 Cross-domain Transformer Encoder는 이종 데이터 형태로 작업할 수 있으므로 보다 유연한 아키텍처이다.

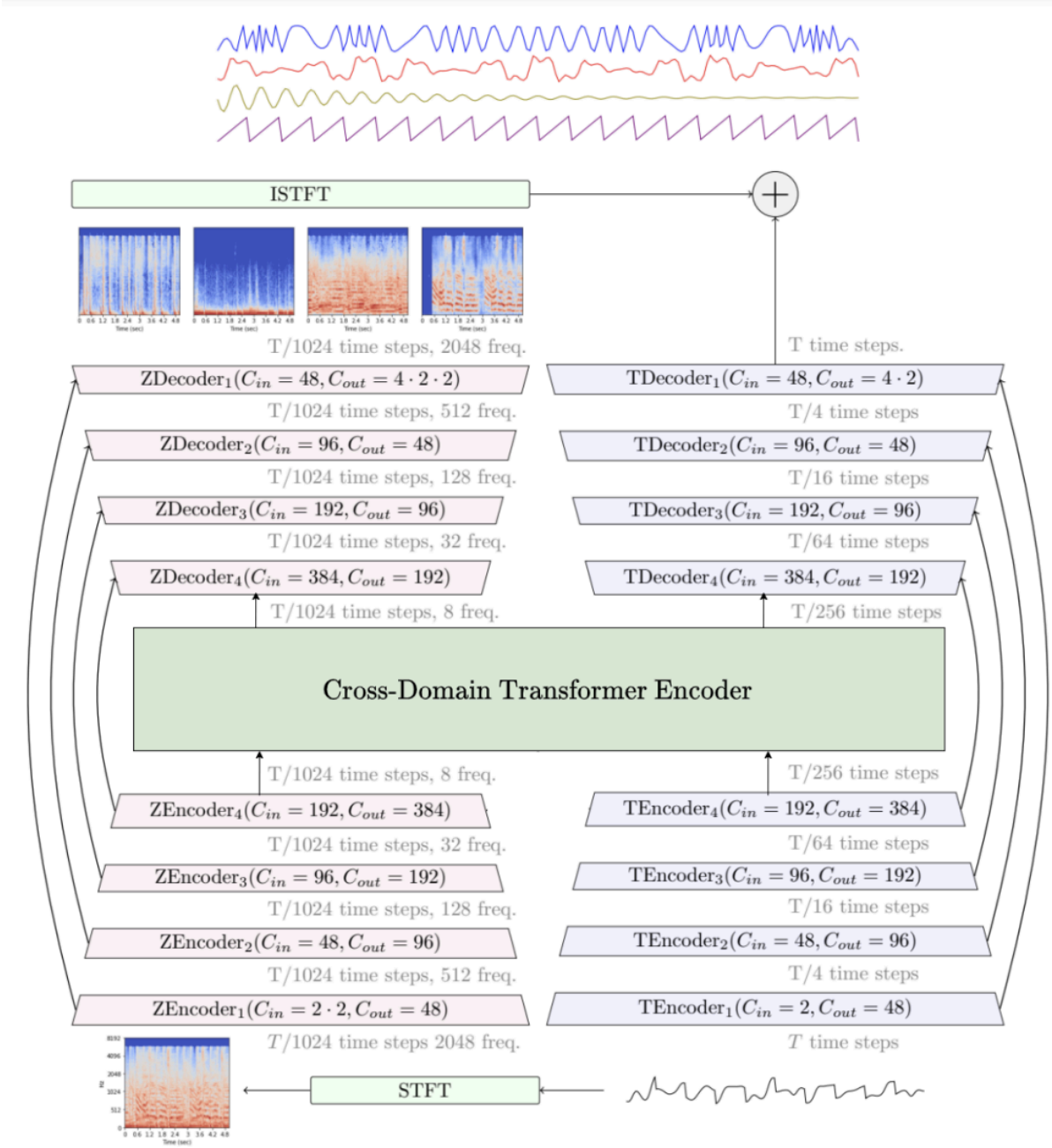


(a) Transformer Encoder Layer

(b) Cross-domain Transformer Encoder of depth 5

Hybrid Transformer Demucs의 아키텍처는 위 그림에 묘사되어 있다. (a)는 Self-Attention 및 Feed-Forward 연산 전에 정규화가 있는 Transformer의 단일 self-attention Encoder layer이며, 학습을 안정화하기 위해 $\epsilon = 10^{-4}$로 초기화된 Layer Scale과 결합된다. 처음 두 개의 정규화는 layer normalization (토큰이 독립적으로 정규화됨)이고 세 번째 정규화는 time layer normalization (모든 토큰이 함께 정규화됨)이다.

Transformer의 입/출력 차원은 384이며 필요에 따라 Transformer 내부 차원으로 변환하기 위해 linear layer를 사용한다. Attention mechanism에는 8개의 head가 있고 feed forward network의 hidden state 크기는 Transfer 차원의 4배와 같다. Cross-attention encoder layer는 동일하지만 다른 도메인 표현과 함께 cross-attention을 사용한다. (b)에는 깊이 5의 Cross-domain Transformer Encoder가 묘사되어 있다. 이것은 스펙트럼 및 파형 도메인에서 self-attention Encoder layer와 cross-attention Encoder layer를 번갈아 쌓은 것이다. 1D 및 2D sinusoidal encoding이 스케일링된 입력에 추가되고 스펙트럼 표현에 reshaping이 적용되어 시퀀스로 처리된다. 아래는 Hybrid Transformer Demucs의 전체 아키텍처이며 2개의 U-Net 인코더/디코더 구조로 구성된다.



메모리 소비와 attention 속도는 시퀀스 길이가 증가함에 따라 빠르게 저하된다. 추가 확장을 위해 sparsity pattern을 동적으로 결정하는 Locally Sensitive Hashing (LSH)와 함께 xformer 패키지의 sparse attention kernel을 활용한다. 90%의 sparsity level (softmax에서 제거된 요소의 비율)을 사용하며, 이는 각각 4개의 bucket으로 LSH를 32회 수행하여 결정된다. Sparsity level이 90%가 되도록 k를 사용하여 LSH의 32회 전체에서 k번 이상 일치하는 요소를 선택한다. 이 변형된 버전을 Sparse HT Demucs라고 한다.

Dataset

저자들은 다양한 음악 장르를 가진 200명의 아티스트의 stem을 가진 3500곡으로 구성된 내부 데이터셋을 선별했다. 각 stem은 음악 프로듀서가 지정한 이름에 따라 4개의 source 중 하나에 지정된다 (ex. “vocals2”, “fx”, “sub” 등). 이 레이블 지정은 이러한 이름이 주관적이고 때로는 모호하기 때문에 noisy하다. 이러한 트랙 중 150개에 대해 자동 레이블 지정이 올바른지 수동으로 확인하고 모호한 stem을 폐기했다. 저자들은 MUSDB와 그 150개 트랙에서 첫 번째 Hybrid Demucs 모델을 학습했다. 여러 규칙에 따라 데이터셋을 전처리한다.

1. 네 가지 source 모두 시간의 30% 이상 침묵하지 않는 stem만 유지한다. 각 1초 세그먼트에 대해 볼륨이 -40dB 미만이면 무음으로 정의한다.
2. $i \in drums, bass, other, vocals$라 하자. 각 stem 및 이전에 언급한 Hybrid Demucs 모델 f를 사용하여 x_i에 주목하는 데이터셋의 노래 x에 대해 $y_{i,j} = f(x_i)_j$, 즉 stem i를 분리할 때 j를 출력한다. 이론적으로 모든 stem에 완벽하게 레이블이 지정되고 f가 완벽한 source separation model인 경우 $y_{i,j} = x_i \delta_{i,j}$ ($\delta_{i,j}$는 Kronecker delta)를 갖게 된다. 파형 z의 경우 1초 세그먼트에 걸쳐 측정된 볼륨을 dB로 정의하였다.

$$V(z) = 10 \cdot \log_{10}(\text{AveragePool}(z^2, 1\text{sec}))$$

Source i, j의 각 쌍에 대해 stem이 있는 1초의 세그먼트를 취하고 $P_{i,j}$를 $V(y_{i,j}) - V(x_i) > -10\text{dB}$인 이러한 세그먼트의 비율로 정의한다. 저자들은 $P \in [0, 1]^{4\times4}$를 얻었고 완벽한 조건에서 $P = Id$를 가져야 함을 알았다. 따라서 모든 source i에 대하여 $P_{i,i} > 0.7$이고 source 쌍 $i \neq j$에 대하여 $P_{i,j} < 0.3$인 노래만 유지한다. 이 과정으로 800곡을 선택한다.

Experiments and Result

- 데이터셋: MUSDB18-HQ (선택된 800 곡)
- Training setup
  - 파형에 대한 L1 loss 사용
  - Adam optimize, no weight decay, learning rate $3 \cdot 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
  - batch size 32, 1200 epochs, 8 Nvidia V100 GPUs

Comparison with the baselines

다음은 MUSDB HQ의 테스트 셋으로 HT Demucs를 baseline들과 비교한 결과이다.

		Test SDR in dB				
Architecture	**Extra?**	All	Drums	Bass	Other	Vocals
IRM oracle	N/A	8.22	8.45	7.12	7.85	9.43
KUIELAB-MDX-Net [17]	✗	7.54	7.33	7.86	5.95	9.00
Hybrid Demucs [2]	✗	7.64	8.12	8.43	5.65	8.35
Band-Split RNN [14]	✗	**8.24**	9.01	7.22	6.70	10.01
HT Demucs	✗	7.52	7.94	8.48	5.72	7.93
Spleeter* [19]	25k	5.91	6.71	5.51	4.55	6.86
D3Net* [12]	1.5k	6.68	7.36	6.20	5.37	7.80
Demucs v2* [10]	150	6.79	7.58	7.60	4.69	7.29
Hybrid Demucs [2]	800	8.34	9.31	9.13	6.18	8.75
Band-Split RNN [14]	1750†	8.97	10.15	8.16	**7.08**	**10.47**
HT Demucs	150	8.49	9.51	9.76	6.13	8.56
HT Demucs	800	8.80	10.05	9.78	6.42	8.93
HT Demucs (fine tuned)	800	9.00	10.08	10.39	6.32	9.20
Sparse HT Demucs (fine tuned)	800	**9.20**	**10.83**	**10.47**	6.41	9.37

Impact of the architecture hyper-parameters

다음은 세그먼트 지속 시간, Transformer 깊이, Transformer 차원의 영향을 나타낸 표이다. OOM은 Out of Memory를 뜻한다.

dur. (sec)	depth	dim.	nb. param.	RTF (cpu)	SDR (All)
3.4	5	384	26.9M	1.02	8.17
3.4	7	384	34.0M	1.23	8.26
3.4	5	512	41.4M	1.30	8.12
7.8	5	384	26.9M	1.49	8.70
7.8	7	384	34.0M	1.68	OOM
7.8	5	512	41.4M	1.77	**8.80**
12.2	5	384	26.9M	2.04	OOM

Impact of the data augmentation

저자들은 더 많은 학습 데이터를 사용하면 augmentation의 필요성이 줄어들기를 기대하였기 때문에 일부 data augmentation을 비활성화할 때의 영향을 실험하였다. 다음은 data augmentation의 영향을 나타낸 표이다.

dur. (sec)	depth	remixing	repitching	SDR (All)
7.8	5	✓	✓	8.70
7.8	5	✓	✗	8.65
7.8	5	✗	✓	8.00

Data augmentation을 비활성화하면 최종 SDR이 지속적으로 저하되는 것을 관찰할 수 있다. Repitching augmentation은 영향이 제한적이지만, remixing augmentation은 모델을 학습시키는 데 여전히 매우 중요하다는 것을 알 수 있다.

Impact of using sparse kernels and fine tuning

Sparse kernel을 테스트하기 위해 깊이를 7로 늘리고 학습 세그먼트 지속 시간을 12.2초로 늘리고 차원 512로 늘렸다. 이 간단한 변경으로 0.14dB의 SDR(8.94dB)이 추가로 생성된다. 학습하는 데 50 epochs만 필요함에도 불구하고 source당 fine-tuning을 통해 SDR이 0.25dB에서 9.20dB로 향상된다고 한다. 저자들은 batch size를 줄임으로써 fine-tuning 단계에서 Transformer Encoder의 receptive field를 15초로 더 확장하려고 시도했지만, 이것은 9.20dB의 동일한 SDR로 이어졌다고 한다.